HAI913I · Évolution et restructuration des logiciels

TP 1 · Annexe 2 — Eclipse JDT et Spoon

Année 2026–2027 Enseignant Bachar Rima

À QUOI SERT CETTE ANNEXE

Donner ce qu'il faut de JDT — et de Spoon, pour la partie bonus — pour réaliser le TP 1, **avec des comportements vérifiés** sur les versions du sujet : JDK 17.0.10 et 21.0.5, JDT Core 3.46.0, Spoon 11.5.0. Les usages de JDT qui ne servent pas au TP (modèle Java d'Eclipse, réécriture d'AST, greffons) sont signalés en fin d'annexe, avec leur documentation officielle.

1. Eclipse JDT Core — parties A et B

1.1. Dépendance

```xml
<dependency>
    <groupId>org.eclipse.jdt</groupId>
    <artifactId>org.eclipse.jdt.core</artifactId>
    <version>3.46.0</version>
</dependency>
```

Cette seule déclaration suffit : Maven télécharge les 18 autres bibliothèques nécessaires (`org.eclipse.core.runtime`, `org.eclipse.core.resources`, `ecj`, `org.eclipse.osgi`…). JDT Core 3.46.0 est compilé pour Java 17 : il ne fonctionne pas sur un JDK plus ancien.

1.2. Configurer `ASTParser`

Le fichier ci-dessous est celui du `starter`, compilé et testé tel quel. Chaque réglage a une raison d'être, que la section suivante mesure.

Listing 1 – `starter/src/main/java/hai913i/tp1/parse/JdtParser.java`

```java
/**
 * Extracteur de modèle (CM 02) : fichiers sources -> AST JDT avec bindings.
 *
 * Chaque réglage est expliqué dans l'annexe « Eclipse JDT et Spoon »
 * (section « Configurer ASTParser »). En retirer un dégrade les bindings,
 * souvent sans erreur visible : vérifiez sur le projet de validation.
 */
public final class JdtParser {

    /** Un fichier source analysé. */
    public record ParsedFile(Path path, CompilationUnit unit) {
    }

    private JdtParser() {
    }

    public static List<ParsedFile> parse(ProjectSources sources,
            List<String> classpath) {
```

```java
        ASTParser parser = ASTParser.newParser(AST.getJLSLatest());

        // Niveau de langage du code analysé : Java 17.
        Map<String, String> options = JavaCore.getOptions();
        JavaCore.setComplianceOptions(JavaCore.VERSION_17, options);
        parser.setCompilerOptions(options);

        parser.setKind(ASTParser.K_COMPILATION_UNIT);
        parser.setResolveBindings(true);
        parser.setBindingsRecovery(true);

        // 1. JAR externes du projet analysé (éventuellement aucun)
        // 2. racine des sources   3. encodage
        // 4. true : inclure le JDK qui exécute l'analyseur (java.lang...)
        parser.setEnvironment(
                classpath.toArray(String[]::new),
                new String[] {sources.sourceRoot().toString()},
                new String[] {"UTF-8"},
                true);

        String[] paths = sources.javaFiles().stream()
                .map(Path::toString)
                .toArray(String[]::new);
        String[] encodings = sources.javaFiles().stream()
                .map(p -> "UTF-8")
                .toArray(String[]::new);

        // Un seul lot : les bindings sont partagés entre les fichiers.
        List<ParsedFile> parsed = new ArrayList<>();
        parser.createASTs(paths, encodings, new String[0],
                new FileASTRequestor() {
                    @Override
                    public void acceptAST(String file, CompilationUnit ast) {
                        parsed.add(new ParsedFile(Path.of(file), ast));
                    }
                }, null);
        return parsed;
    }
}
```

RÉGLAGE	RÔLE
`newParser(AST.getJLSLatest())`	niveau d'API de l'AST : le plus récent connu de cette version de JDT (26 pour 3.46.0).
`setComplianceOptions(VERSION_17)`	niveau de langage du code *analysé*.
`setKind(K_COMPILATION_UNIT)`	l'entrée est un fichier complet (et non une expression ou une suite d'instructions).
`setResolveBindings(true)`	calculer les *bindings* : types, cibles des appels, superclasses.
`setBindingsRecovery(true)`	produire des *bindings* « récupérés » quand un type est introuvable, au lieu de `null`.
`setEnvironment(cp, src, enc, true)`	où chercher les types : JAR du projet, racine des sources, encodage, JDK courant.
`createASTs(. . .)`	analyser tous les fichiers en un lot, avec un environnement de résolution partagé.

1.3. Mesures : que se passe-t-il si l'on s'écarte de cette configuration ?

Le projet de validation contient 93 `MethodInvocation`. Le tableau indique, pour chaque variante, combien reçoivent un *binding* de méthode. Mesures identiques sous JDK 17.0.10 et 21.0.5.

	VARIANTE	RÉSULTAT MESURÉ
A	configuration de la section 1.2 (`createASTs`)	**93/93** ; 1 avertissement (`serialVersionUID`)
B	un fichier à la fois, `setSource(char[])` et `setUnitName("Book.java")`	93/93
C	idem, mais `setUnitName("")` (configuration de l'ancienne correction)	93/93, mais **15 problèmes** : « The public type … must be defined in its own file »
D	sans `setEnvironment`	**exception** `IllegalStateException: Missing system library`
E	`setEnvironment(. . ., false)` sans JDK dans le *classpath*	même exception
F	`createASTs` sans racine des sources	93/93 : tous les fichiers sont dans le même lot
G	sans `setResolveBindings(true)`	**0/93**
H	un fichier à la fois, racine des sources = dossier du projet au lieu de `src`	**49/93**, sans aucune exception

DEUX PANNES SILENCIEUSES

H ne lève aucune erreur : la moitié des appels deviennent simplement non résolus. **Le niveau de langage** peut aussi faire perdre du code sans rien signaler : avec `AST.JLS8`, la déclaration `record P(int x) {}` donne **un fichier sans aucun type et zéro problème** ; avec `AST.JLS17` ou `getJLSLatest()` et les options de la section 1.2, un `RecordDeclaration`. Sans `setCompilerOptions`, la même ligne est lue comme une construction de Java 25 et signalée en erreur. D'où la règle : vérifiez toujours vos comptes sur le projet de validation.

1.4. Les nœuds utiles au TP

CONSTRUCTION JAVA	NŒUD JDT	À SAVOIR
fichier	`CompilationUnit`	`getPackage()` (`null` pour le paquetage par défaut), `types()`, `getProblems()`
classe, interface	`TypeDeclaration`	`isInterface()`, `isMemberTypeDeclaration()`, `isLocalTypeDeclaration()`
énumération	`EnumDeclaration`	**pas** un `TypeDeclaration` ; constantes : `enumConstants()`
record	`RecordDeclaration`	**pas** un `TypeDeclaration` ; composants : `recordComponents()`
membres d'un type	`bodyDeclarations()`	commun à ces trois nœuds (`AbstractTypeDeclaration`)
attribut	`FieldDeclaration`	plusieurs variables : `fragments()` de `VariableDeclarationFragment`
méthode, constructeur	`MethodDeclaration`	`isConstructor()`, `parameters()`, `getBody()` (`null` si abstraite)
`a.m()`, `m()`, `T.m()`	`MethodInvocation`	`getExpression()` : le receveur, `null` si implicite
`super.m()`	`SuperMethodInvocation`	nœud distinct de `MethodInvocation`
`new T(. . .)`	`ClassInstanceCreation`	`getAnonymousClassDeclaration()` si classe anonyme
`this(. . .)`, `super(. . .)`	`ConstructorInvocation`, `SuperConstructorInvocation`	instructions, en tête de constructeur
lambda	`LambdaExpression`	son corps est visité comme le reste de la méthode
`T::m`	`ExpressionMethodReference`…	**pas** un `MethodInvocation`

1.5. Parcourir : `ASTVisitor`

`node.accept(visiteur)` parcourt le sous-arbre en profondeur. Pour chaque nœud, JDT appelle `preVisit2(node)`, puis la méthode `visit` du type concret ; si elle renvoie `true`, les enfants sont visités ; viennent ensuite `endVisit` puis `postVisit`. Renvoyer `false` **élague** le sous-arbre : c'est ainsi qu'on ignore le corps d'une classe anonyme. L'annexe 3 relie ce mécanisme au patron Visitor et en donne un exemple exécutable.

1.6. Positions et lignes

`node.getStartPosition()` et `node.getLength()` sont des positions en **caractères** ; `unit.getLineNumber(position)` les convertit en lignes. Deux pièges mesurés :

- la position d'une `MethodDeclaration` **commence à sa Javadoc** et inclut ses annotations. Sur une méthode précédée de trois lignes de Javadoc et d'un `@Override`, la déclaration commence ligne 20, le nom est ligne 24 et le corps occupe les lignes 24 à 26 ;
- la dernière ligne d'un nœud est `getLineNumber(getStartPosition() + getLength() - 1)` : sans le `- 1`, on compte parfois la ligne suivante.

Pour la convention du TP (lignes du *corps*), mesurez `getBody()`.

1.7. *Bindings* : ce que l'AST seul ne dit pas

L'AST est syntaxique : il contient `extends Item`, pas la chaîne complète des superclasses, ni la déclaration visée par `loanable.checkOut(member)`. Les *bindings* apportent ces informations sémantiques.

APPEL	RÉSULTAT (MESURÉ)
`typeDecl.resolveBinding()`	`ITypeBinding` du type déclaré
`binding.getSuperclass()`	remonter jusqu'à `java.lang.Object` : `LoanException` → `Exception` → `Throwable` → `Object`
`binding.getQualifiedName()`	`library.service.LoanService.Loan` ; **chaîne vide** pour une classe anonyme ou locale
`binding.getBinaryName()`	`library.service.LoanService$Loan` ; `demo.Square$1` pour une classe anonyme
`binding.getErasure()`	`java.util.List` au lieu de `java.util.List<library.model.Item>`
`invocation.resolveMethodBinding()`	`IMethodBinding` de la cible, choisie d'après le type statique du receveur
`methodBinding.getDeclaringClass()`	pour `zadig.getTitle()` : `Item` (où la méthode est déclarée)
`methodBinding.getMethodDeclaration()`	la déclaration générique : `forEach` de `Iterable<Shape>` devient celui d'`Iterable`
`expression.resolveTypeBinding()`	type statique du receveur : `Loanable` pour `loanable`, `Book` pour `zadig`
`typeBinding.isFromSource()`	`true` si le type vient des sources analysées (utile pour « interne / externe »)
`binding.isRecovered()`	`true` pour un *binding* reconstruit faute d'information

1.8. Quand un *binding* est nul

1. `setResolveBindings(true)` a-t-il été appelé *avant* `createAST(s)` ?
2. La racine des sources est-elle le dossier qui *contient les paquetages* (`src/main/java` ou `src`), et non le dossier du projet (variante H) ?
3. Le type appartient-il à une **bibliothèque absente** ? Sur `resources/robustness/missing-dep`, JDT signale « The import org.apache cannot be resolved » : `StringUtils.capitalize(name)` n'a ni cible ni type de receveur, mais `formatter.decorate(clean)`, dans la même méthode, est résolu. Ajoutez les JAR du projet analysé au premier argument de `setEnvironment` si vous les avez ; sinon, comptez l'appel comme non résolu.

4. Le fichier **compile-t-il** ? Sur `syntax-error`, la méthode fautive est marquée `ASTNode.MALFORMED` et l'instruction en erreur n'apparaît pas dans l'AST ; l'autre fichier est analysé normalement.
5. Affichez `unit.getProblems()` : le message dit presque toujours ce qui manque.

1.9. Constructions récentes (mesuré)

CONSTRUCTION	CE QUE PRODUIT JDT 3.46.0 (NIVEAU JAVA 17)
record	`RecordDeclaration`, ignoré par `visit(TypeDeclaration)`
interface `sealed ... permits`	`TypeDeclaration` (`isInterface()`)
`o instanceof Circle c`	*binding* résolu : `c.r()` vise `Circle#r`
appel dans une lambda	`MethodInvocation` visité normalement ; le type du paramètre de lambda est inféré (`s.area()` vise `Shape#area`)
référence de méthode	`ExpressionMethodReference`, jamais `MethodInvocation`
classe anonyme	`AnonymousClassDeclaration` ; ses méthodes sont des `MethodDeclaration` dont le type déclarant n'a pas de nom qualifié
classe locale	`TypeDeclaration` avec `isLocalTypeDeclaration()`
import statique (`max(...)`)	`MethodInvocation` sans expression ; cible `java.lang.Math#max`
lambda dans un initialiseur de champ	ses appels ne sont dans aucune `MethodDeclaration`

2. Spoon — partie C (bonus)

Spoon construit un **métamodèle** du programme (interfaces `Ct...`) au-dessus du compilateur d'Eclipse. Il offre des requêtes plus concises que `ASTVisitor`, mais ses conventions de comptage diffèrent de celles du TP.

2.1. Dépendance et construction du modèle

```
<dependency>
    <groupId>fr.inria.gforge.spoon</groupId>
    <artifactId>spoon-core</artifactId>
    <version>11.5.0</version>
</dependency>
```

Spoon 11.5.0 dépend de JDT Core 3.46.0 : si vous utilisez les deux, une seule version de JDT se retrouve sur le *classpath* (vérifié avec `mvn dependency:tree`). Sans implémentation SLF4J, Spoon affiche un avertissement au démarrage ; la dépendance `org.slf4j:slf4j-nop:2.0.18` le supprime.

Listing 2 – Configuration du `Launcher` utilisée pour les mesures (extrait de `SpoonSpike.java`, interne)

```
84      static Launcher launcher(Path src, boolean noClasspath) {
85          Launcher launcher = new Launcher();
86          launcher.addInputResource(src.toString());
87          launcher.getEnvironment().setNoClasspath(noClasspath);
88          launcher.getEnvironment().setComplianceLevel(17);
89          launcher.getEnvironment().setCommentEnabled(false);
90          launcher.getEnvironment().setShouldCompile(false);
91          Path out = Paths.get(System.getProperty("java.io.tmpdir"), "spooned");
92          launcher.setSourceOutputDirectory(out.toString());
93          launcher.getEnvironment().setOutputType(spoon.OutputType.NO_OUTPUT);
94          return launcher;
95      }
```

API	À SAVOIR
`launcher.buildModel()` → `CtModel`	le modèle de tout le projet
`model.getAllTypes()`	types de **premier niveau** seulement (14 sur le projet de validation)
`model.getElements(new TypeFilter<>(CtType.class))`	tous les types, membres compris (15)
`type.getMethods()`	méthodes **sans** les constructeurs ; `CtClass.getConstructors()` à part
`type.getFields()`	inclut les **constantes d'énumération** (`CtEnumValue`)
`CtInvocation.getTarget()`	le receveur (`CtThisAccess` implicite, `CtTypeAccess` pour un appel statique, `null` pour `super(. . .)`)
`invocation.getExecutable()`	référence de la cible ; `getExecutableDeclaration()` donne sa déclaration
`element.getPosition()`	`getLine()`, `getEndLine()` ; tester `isValidPosition()` : un élément implicite n'a pas de position
`type.getQualifiedName()`	`library.service.LoanService$Loan` (avec `$`)

2.2. *noClasspath* ou *classpath* complet

En mode *noClasspath* (`setNoClasspath(true)`), Spoon construit un modèle même si des types sont introuvables, en devinant ce qu'il peut. Sur `missing-dep`, il attribue `capitalize` à `org.apache.commons.lang3.StringUtils`, déduit de l'`import`, sans que la bibliothèque existe. Avec `setNoClasspath(false)`, notre essai de `buildModel()` n'a pas levé d'exception : les erreurs sont journalisées par SLF4J, donc invisibles sans journaliseur configuré. Dans les deux cas, ne concluez pas qu'un appel est « résolu » sans l'avoir vérifié.

2.3. Filtres, requêtes, processeurs

Trois façons de parcourir un modèle Spoon : `CtScanner` (le visiteur de Spoon), `getElements(filtre)` et `filterChildren(. . .).map(. . .).list()` (des requêtes), et `AbstractProcessor<E>` (un traitement appliqué par Spoon à chaque élément d'un type). L'annexe 3 les montre sur un exemple exécutable ; `resources/spoon-tostring` utilise un processeur pour *transformer* du code.

2.4. JDT et Spoon sur le projet de validation

MESURE	CONVENTION DU TP	SPOON 11.5.0, BRUT (MESURÉ)
types	15	`getAllTypes()` : 14 ; tous les `CtType` : 15
méthodes	59	50 `CtMethod` + 13 `CtConstructor`, dont 4 implicites
attributs	21	24 `CtField` : les 3 constantes de `Category` en plus
paquetages	4	`getAllPackages()` : 6 (paquetage racine et `library`)
appels	94	107 `CtInvocation` : 13 appels `super(. . .)` de constructeurs, dont les implicites
graphe d'appel	59 / 44 / 46	identique (nœuds, arcs, sites) une fois les conventions du TP appliquées

POURQUOI JDT EST REQUIS ET SPOON EN BONUS

JDT est l'outil du CM 02, il oblige à comprendre la configuration de la résolution de noms, et ses nœuds correspondent directement aux questions du TP. Spoon épargne beaucoup de code, mais ses choix (constructeurs implicites, constantes d'énumération, mode *noClasspath*) masquent précisément les questions de définition que le TP veut faire poser. Le comparer aux résultats JDT est donc un bon exercice de bonus.

3. Ressources en ligne

Liens vérifiés le 14 septembre 2026.

RESSOURCE	STATUT	COMMENTAIRE
Javadoc JDT Core : ASTParser, ASTVisitor, CompilationUnit, AST	obligatoire	référence officielle, version courante
Guide JDT : Manipulating Java code	utile	officiel ; une partie concerne les greffons Eclipse
Article Eclipse Corner : Abstract Syntax Tree	utile	concepts toujours justes ; API de 2006
vogella — Eclipse JDT	facultatif	orienté greffons (modèle Java d'Eclipse)
AST View	facultatif	greffon Eclipse d'exploration d'AST
Dépôt JDT Core	facultatif	sources et suivi des versions
Documentation Spoon (lanceur, éléments, références, filtres, processeurs)	utile (bonus)	officielle
Dépôt Spoon	facultatif	sources, exemples
R. Pawlak *et al.*, « Spoon : A Library for Implementing Analyses and Transformations of Java Source Code », *Software : Practice and Experience*, 2015, HAL	facultatif	l'article de référence

CE QUE CETTE ANNEXE NE COUVRE PAS

Le modèle Java d'Eclipse (`IJavaProject`, espace de travail), les descripteurs de propriétés structurelles, la réécriture d'AST (`ASTRewrite`), et, pour Spoon, les chemins `CtPath`, gabarits, motifs, générateurs, lanceur incrémental et analyse de *bytecode*. Ces mécanismes ne servent pas au TP 1 ; la documentation officielle ci-dessus les décrit dans leur version actuelle.